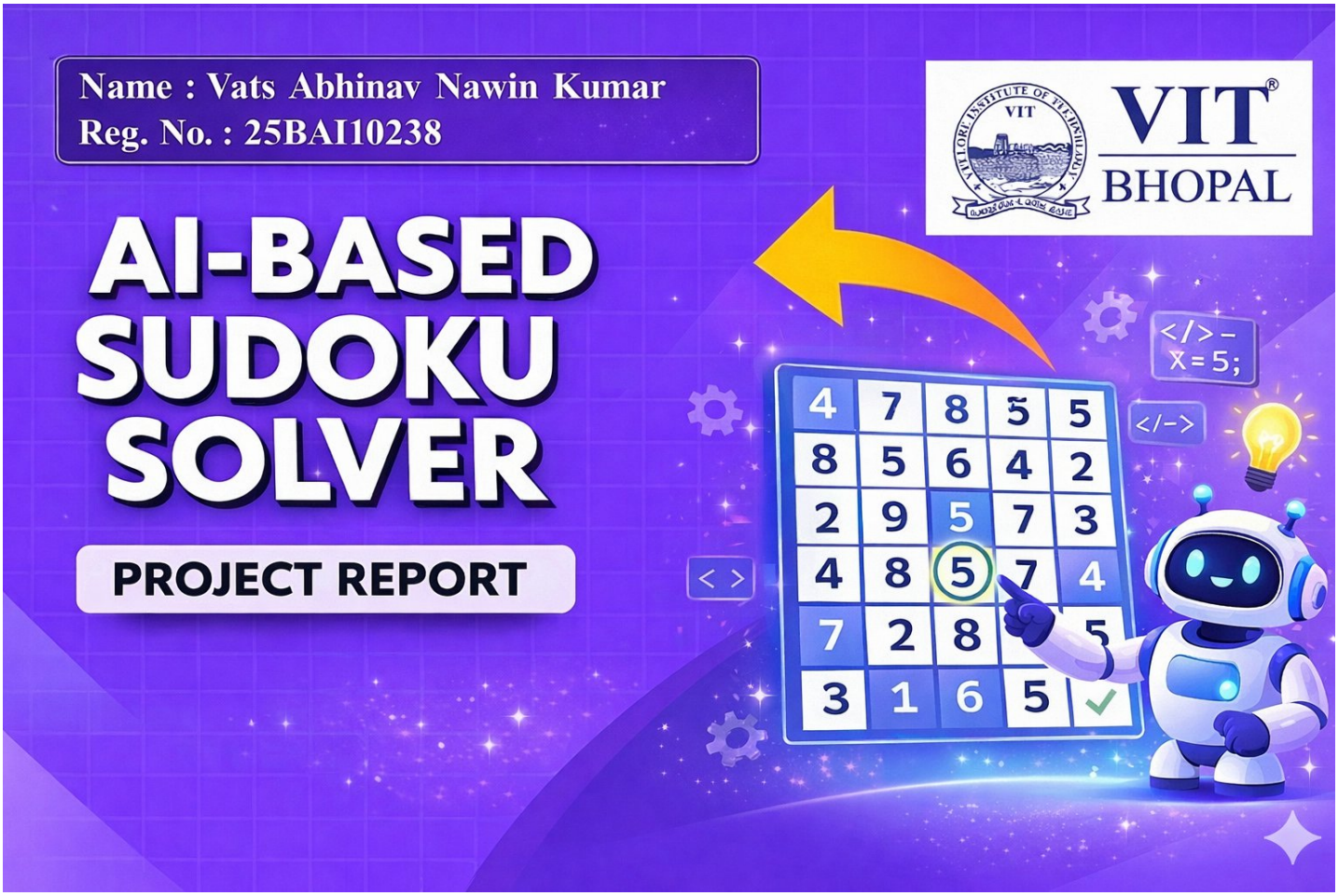

Project Report: AI-Based Sudoku Solver

Date: March 28,2026

Subject: Fundamentals of AI and ML (CSA2001)

AI-BASED SUDOKU SOLVER

A Console-Based Application using Constraint Satisfaction

Abstract: This project aims to develop a console-based Sudoku Solver using Python. The system takes a Sudoku puzzle as input, validates it and solves it using a constraint satisfaction approach. The application ensures that all Sudoku rules are followed and provides a correct solution efficiently.

2. Introduction

Sudoku is a popular logic-based puzzle that requires filling a 9×9 grid such that each row, column and 3×3 subgrid contains numbers from 1 to 9 without repetition. Solving Sudoku manually can be time-consuming and error-prone.

This project implements a Python-based Sudoku solver that automates the solving process using constraint satisfaction techniques. The system also validates user input to ensure correctness before solving.

3. Problem Statement

Solving Sudoku puzzles manually can be difficult, especially for complex puzzles. Users may also enter incorrect inputs without realizing mistakes.

There is a need for a:

1. Simple tool to input Sudoku puzzles easily.
2. System that validates input to prevent errors.
3. Method to automatically solve Sudoku puzzles.
4. Reliable and efficient approach to ensure correct solutions.

4. Functional Requirements

The system fulfills the following functional requirements:

- REQ-1 (Input): The system must allow users to enter Sudoku row by row.
- REQ-2 (Validation): The system must check for duplicate values and invalid inputs.
- REQ-3 (Processing): The system must solve the Sudoku using constraint satisfaction.
- REQ-4 (Output): The system must display the solved Sudoku clearly.
- REQ-5 (Error Handling): The system must detect invalid Sudoku inputs.

5. Non-functional Requirements

Usability: The interface must be simple and easy to use.

Performance: The solution should be generated quickly.

Portability: The program should run on any system with Python installed.

Reliability: The system must produce correct results for valid inputs.

6. System Architecture

The application follows a simple procedural architecture:

- **Input Layer:** Handles user input using console prompts.
- **Logic Layer:** Implements validation and solving logic using constraint satisfaction.
- **Data Layer:** Stores Sudoku grid as a 2D list.

7. Design Diagrams

7.1 Use Case Diagram

- Actor: User

- Use Cases:
  - Enter Sudoku
  - Validate Input
  - Solve Sudoku
  - Display Solution

7.2 Workflow

1. Start program
2. Take input row by row
3. Validate each row
4. After full input, validate grid
5. Solve using constraint satisfaction
6. Display result
7. End

7.3 Sequence Diagram (Solving Process)

User → System: Enter Sudoku

System → Validation Module: Check input

Validation Module → System: Valid / Invalid

If valid:

System → Solver: Apply constraints

Solver → System: Solution

System → User: Display solution

7.4 Component Structure

1. Input Module → Handles user input
2. Validation Module → Checks correctness
3. Solver Module → Uses constraint satisfaction
4. Output Module → Displays Sudoku

7.5 Data Structure

Data Structure Used: 2D List

Grid: puzzle[9][9]

Each element represents a cell value (0 for empty, 1–9 for filled)

8. Design Decisions & Rationale

- **Language (Python):** Chosen for simplicity and readability.
- **Constraint Satisfaction:** Used because Sudoku is a constraint-based problem.
- **CLI Interface:** Chosen for simplicity and ease of implementation.
- **2D List:** Efficient for representing Sudoku grid.

9. Implementation Details

The implementation consists of:

1. Input Handling: Users enter Sudoku row by row using input().

2. Validation Functions: Check rows and columns for duplicates.

3. Solver Function: Uses python-constraint library to apply constraints and find a solution.

4. Output: Displays Sudoku in formatted grid.

10. Screenshots / Results

1. Input Start:

```
PS D:\Program Files 2\Microsoft VS Code> & "d:\New Downloads\venv\Scripts\python.exe" c:/Users/ABHINAV/Desktop/Program/MyPy.py
Enter Sudoku row by row (Use 0 for empty cells):
Please enter space between the numbers
Row 1:
```

2. Invalid Row:

```
Row 1: 1 2 3 4 5 5 6 7 8

Invalid row. Duplicate values found.
```

3. Current Sudoku Display :

```
Current sudoku:
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
---------------------
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
---------------------
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
```

4. Final Input Sudoku:

```
Final input sudoku:

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
---------------------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
---------------------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

5. Solved Sudoku:

```
Solved sudoku:

5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
---------------------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
---------------------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

11. Testing Approach

Testing was done using different inputs:

Test Case ID | Description | Expected Result | Status

TC-01 | Valid Sudoku | Correct solution | Pass

TC-02 | Duplicate in row | Error message | Pass

TC-03 | Column conflict | Invalid Sudoku | Pass

TC-04 | Incorrect input size | Error message | Pass

TC-05 | Empty cells | Solved correctly | Pass

12. Challenges Faced

1. Handling invalid inputs correctly.
2. Managing constraint logic for solving Sudoku.
3. Ensuring no duplicates in rows and columns.
4. Formatting output for better readability.

13. Learnings & Key Takeaways

- Learned constraint satisfaction concepts.
- Improved understanding of problem-solving.
- Gained experience with Python libraries.
- Practiced input validation techniques.

14. Future Enhancements

1. Add GUI interface.
2. Add Sudoku generator.
3. Add difficulty levels.
4. Visualize solving steps.

15. References

1. Python Documentation
2. Python-constraint Library
3. Sudoku solving concepts